

Երևանի Պետական Համալսարան

Spotify Տվյալների Բազայի Վերլուծություն Վիճակագրական Մեթոդներով

Ուսանող՝ Արթուր Փիլիպոսյան
Դասընթաց՝ Բազմաչափ վիճակագրություն
Դասախոս՝ Ա. Դավայան

1 Ներածություն

Անձնական հետաքրքրությունից ելնելով՝ բազմաչափ վիճակագրությունն առարկայից նախագծային աշխատանքի համար ընտրել եմ Spotify երաժշտական սթրիմինգային ընկերության տվյալների բազան: Այս տվյալների բազան տրամադրում է երաժշտության լայն նկարագրություն մի քանի կատեգորիական և քանակական բնութագրերով, ինչպիսիք են երգերի էներգետիկան, ռիթմիկությունը, ակուստիկությունը, տեմպը և այլն: Նախագծի նպատակն է ուսումնասիրել Spotify տվյալներում առկա հատկանիշների միջև կապերը, ստուգել, թե արդյոք փոփոխականներն ունեն գաուսյան բաշխումներ, գծային ռեգրեսիայի միջոցով կանխատեսել երգերի հանրաճանաչությունը և հիմնական բաղադրիչների վերլուծություն (ՀԲՎ) իրականացնել դրանց վրա:

2 Տվյալների Նկարագրություն և Մոտիվացիա

Տվյալների բազան վերցված է Kaggle կայքից¹: Այն պարունակում է 114,000 տող, ներկայացված 20 հատկանիշային սյուներով: Ներկայացված են 1,000-ական երգեր՝ 114 ժանրային խմբերից: Տվյալների բազան հավասարակշռված է ըստ ժանրերի. այն պակաս-պատահական նմուշ է Spotify ընկերության տվյալներից: Այնուամենայնիվ, տվյալները ունեն նախամշակման կարիք: Սկզբնական բազայից դուրս են հանվել 24,260 կրկնվող տողեր: Տողերում դատարկ թողնված տվյալներ չկային: Վերջնական վերլուծությունում օգտագործվել է **89,740 երաժշտության նմուշ**:

Ուսումնասիրության նպատակից ելնելով՝ օգտագործվել են տվյալների բազայի քանակական փոփոխականները. դուրս են թողնվել երգի ինդեքսը, կատարողի և երգի անունը, տոնայնությունը, մինոր/մաժոր հատկանիշը, ինչպես նաև ժանրը: Աղյուսակ 1-ում նկարագրված են վերլուծության մեջ ներառված փոփոխականները: Հիմնական հետաքրքրությունը ներկայացնում է հանրաճանաչելիության հատկանիշը, որը և գծային ռեգրեսիայի նպատակային փոփոխականն է լինելու:

Աղյուսակ 1: Ընտրված քանակական փոփոխականները և դրանց նկարագրությունը:

Փոփոխական	Նշանակություն	Միջին	Ստ. շեղում
popularity	Հանրաճանաչելիության միավորը՝ 0-ից 100	33.20	20.58
duration_ms	Երգի տևողությունը միլիվայրկյաններով	229,144	112,946
danceability	Պարայինության գործակիցը՝ 0-ից 1	0.562	0.177
energy	Էներգետիկությունը՝ 0-ից 1	0.634	0.257
loudness	Միջին բարձրությունը դեցիբելներով	-8.499	5.222
speechiness	Խոսքային հատվածների առկայությունը՝ 0-ից 1	0.087	0.113
acousticness	Երգի ակուստիկ լինելու գործակիցը՝ 0-ից 1	0.328	0.338
instrumentalness	Ինստրումենտալ լինելու գործակիցը՝ 0-ից 1	0.173	0.324
liveness	Կենդանի կատարմամբ լինելու գործակիցը՝ 0-ից 1	0.217	0.195
valence	Դրական լիցքերը	0.469	0.263
tempo	Երաժշտական տեմպը	122.06	30.12

Աղյուսակ 1-ը ցույց է տալիս երգերի միջև զգալի տարբերություն: Միջին տևողությունը

¹<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

մոտ 3.82 րոպե է, իսկ տևողության մեծ ստանդարտ շեղումը պայմանավորված է մի քանի շատ երկար երգերով: Պարայինության և էներգետիկության միջինները ցույց են տալիս, որ տիպիկ երգը չափավոր պարային և էներգետիկ է, սակայն ստանդարտ շեղումները հաստատում են, որ տվյալներում կան բազմազան երաժշտական ոճեր:

3 Ընտրանքի նկարագրական վերլուծություն

Պատկերներն ու գրաֆիկները տեղադրված են Հավելված A-ում: Նկար 1-ում կարող ենք տեսնել մի քանի կարևոր փոփոխականների միջև ցրվածության զույգ առ զույգ դիագրամը: Ամենահստակ աճող կապը էներգետիկության և երգի բարձրության միջև է. էներգետիկ երգերը սովորաբար ավելի բարձր են: Ակուստիկությունը տեսանելի նվազող կապ ունի թե՛ էներգետիկության, և թե՛ ձայնի բարձրության հետ: Պարայինությունն ու դրական լիցքերը նույնպես ունեն որոշակի դրական կապ: Ի տարբերություն դրանց՝ հանրաճանաչելիության գրաֆիկները ցրված են և որևէ ձայնային հատկանիշի հետ ուժեղ գծային կապ ցույց չեն տալիս:

Նկար 2-ում կորելացիոն ջերմային քարտեզը հաստատում է վերոնշյալ օրինաչափությունները: Էներգետիկությունն ու ձայնի բարձրությունն ունեն ամենաուժեղ դրական կորելացիան՝ $r = 0.76$, էներգետիկությունն ու ակուստիկությունը ուժեղ բացասական կորելացված են՝ $r = -0.73$, իսկ բարձրությունն ու ակուստիկությունը ևս ունեն բացասական կապ՝ $r = -0.58$: Պարայինության և դրական հնչողության կորելացիան չափավոր դրական է՝ $r = 0.49$: Հանրաճանաչության ամենամեծ բացարձակ կորելացիան ընդամենը $r = -0.13$ է՝ ինստրումենտալության հետ: Հետևաբար ճանաչելիությունը գծայնորեն կախված չէ ներառված փոփոխականներից:

4 Գաուսյան Բաշխման Վերլուծություն

Գրեթե 90,000 դիտարկման դեպքում գործնականորեն նպատակահարմար է օգտագործել Q-Q գրաֆիկների մեթոդը՝ ստուգելու համար արդյոք տվյալները գալիս են նորմալ բաշխումից, թե ոչ:

Նկար 3-ում պատկերված են հանրաճանաչելիության, պարայինության, ակուստիկության, ինստրումենտալության և երաժշտության տեմպի Q-Q գրաֆիկները: Այս մեթոդով ստուգվում են թե արդյոք գործնական տվյալների քվանտիլները համապատասխանում են իրական նորմալ բաշխման քվանտիլներին, թե ոչ: Պատկերներից նկատենք, որ ներկայացված փոփոխականներից ոչ մեկը ճշգրիտ Գաուսյան բաշխում չունի: Հանրաճանաչության բաշխումն ունի անկանոն տեսք և մեծամասամբ կուտակված է զրոյի մոտ, թեև ընդհանուր շեղվածությունը փոքր է: Պարայինության բաշխման կենտրոնական մասը մոտ է զանգակաձևին, սակայն փոփոխականը սահմանափակված է 0-ից 1 միջակայքում, ուստի պոչերը չեն կարող նորմալ բաշխման նման լինել: Էներգետիկությունը, ակուստիկությունը և ինստրումենտալությունը եզրերում կտրուկ կուտակում ունեն: Տեմպ փոփոխականը ամենամոտն է Գաուսյան տեսքին, սակայն ունի մի քանի գագաթ և պոչային շեղումներ: Արդյունքում ամբողջ տվյալների համար բազմաչափ Գաուսյան ենթադրությունը

իրատեսական չէ:

5 Գծային Ռեգրեսիա

Որպես ռեգրեսիայի բացատրվող փոփոխական վերցնենք հանրաճանաչելիությունը, այն ունկնդիրների նախընտրության կարևոր ցուցիչն է: Անկախ փոփոխականներն են երաժշտության տևողությունը (`duration_ms`), պարայինությունը (`danceability`), էներգետիկությունը (`energy`), ձայնի բարձրությունը (`loudness`), խոսքային մասերի առկայությունը (`speechiness`), ակուստիկությունը (`acousticness`), գործիքային լինելը (`instrumentalness`), կենդանի կատարման լինելու գործակիցը (`liveness`), դրական լիցքերը (`valence`) և տեմպը (`tempo`): Մոդելի կառուցումից առաջ անկախ փոփոխականները ստանդարտացվել են: Արդյունքում յուրաքանչյուր գործակից ցույց է տալիս այլ հավասար պայմաններում հանրաճանաչության արժեքի ակնկալվող փոփոխությունը տվյալ հատկանիշի մեկ ստանդարտ շեղմամբ աճի դեպքում՝ մյուս հատկանիշները հաստատուն պահելով:

Աղյուսակ 2: Հանրաճանաչության գծային ռեգրեսիան՝ ստանդարտացված անկախ փոփոխականներով:

Անկախ փոփոխական	Գործակից	<i>p</i> -արժեք
Ազատ անդամ	33.199	< 0.001
<code>duration_ms</code>	-0.455	< 0.001
<code>danceability</code>	1.825	< 0.001
<code>energy</code>	-0.547	< 0.001
<code>loudness</code>	0.479	< 0.001
<code>speechiness</code>	-1.383	< 0.001
<code>acousticness</code>	-0.554	< 0.001
<code>instrumentalness</code>	-2.896	< 0.001
<code>liveness</code>	0.102	0.159
<code>valence</code>	-2.185	< 0.001
<code>tempo</code>	0.150	0.034

Մոդելի համար $R^2 = 0.0309$, իսկ ճշգրտված $R^2 = 0.0308$ է: Ընտրված ձայնային հատկանիշները բացատրում են հանրաճանաչության տատանման միայն մոտ 3.1%-ը: Պարայինության գործակիցը դրական է. նրա մեկ ստանդարտ շեղմամբ աճը հանգեցնում է հանրաճանաչության մոտ 1.83 միավոր աճի: Ամենամեծ բացասական գործակիցն ունի գործիքային լինելը, որին հաջորդում են դրական հնչողությունը և խոսքային լինելը: Ակուստիկությունը ևս ունի փոքր բացասական գործակից, ինչը մյուս փոփոխականները ֆիքսելուց հետո ցուցաբերում է փոքր-ինչ ցածր հանրաճանաչության միտում:

Նախորդ վերլուծությունից հիշենք, որ էներգետիկությունն ու դրական լիցքեր հաղորդելու հակումը կորելացված են, հետևաբար դրանց գործակիցները նկարագրում են միմյանց վրա պայմանական կապերը, ոչ թե պարզ ազդեցություններ: Մոդելը նկարագրական է և չի ապացուցում, որ ձայնային հատկանիշի փոփոխությունը պատճառում է հանրաճանաչության փոփոխություն: Ցածր R^2 -ից հետևում է, որ երգի հայտնիության վրա շատ այլ գործոններ են ազդում, քան

միայն երաժշտական տվյալները, օրինակ՝ կատարողի հեղինակությունը, գովազդը, թողարկման պահը և այլն:

6 Փոփոխականների Ընտրություն

Այն փոփոխականները, որոնց p -արժեքները մեծ են՝ 5% նշանակալիության մակարդակից, հեռացվել են հետագա հետազոտությունից: Օրինակ՝ liveness-ն ունեցել է ամենամեծ p -արժեքը ($p = 0.159$) և հեռացվել է: Մնացած փոփոխականների p -արժեքները 0.05-ից փոքր են՝ տևողություն, պարայինություն, էներգետիկություն, բարձրություն, խոսքային լինել, ակուստիկություն, գործիքային լինել, դրական հնչողություն և տեմպ:

Պարզեցված մոդելի համար $R^2 = 0.03084$ է, ինչը գրեթե չի տարբերվում նախորդ մոդելի $R^2 = 0.03087$ արժեքից: Այսպիսով, liveness-ի հեռացումը մոդելը պարզեցնում է՝ առանց բացատրելի փոփոխականի մեկնաբանման կարողությունը նվազեցնելու: Մյուս կողմից՝ շատ մեծ նմուշի դեպքում փոքր ազդեցությունները ևս դառնում են վիճակագրորեն նշանակալի, ուստի ընդհանուր ցածր R^2 -ը ավելի կարևոր է, քան միայն p -արժեքները:

7 Հիմնական Բաղադրիչների Վերլուծություն

ՀԲՎ-ը կիրառվել է ընտրված բոլոր 11 քանակական փոփոխականների, ներառյալ հանրաճանաչության, նկատմամբ: Փոփոխականները նախապես ստանդարտացվել են:

Աղյուսակ 3: Առաջին 5 հիմնական բաղադրիչների բացատրած վարիացիան:

Բաղադրիչ	Բացատրած վարիացիա	Կուտակված վարիացիա
ՀԲ1	26.33%	26.33%
ՀԲ2	13.88%	40.21%
ՀԲ3	11.53%	51.74%
ՀԲ4	9.52%	61.25%
ՀԲ5	8.68%	69.93%

Աղյուսակ 3-ում տեսնում ենք, որ առաջին երկու բաղադրիչներն ամփոփում են ստանդարտացված ընդհանուր տատանման 40.21%-ը: Սա մեծամասամբ բավարար է տվյալների վիզուալիզացիայի համար, սակայն դրանք դեռ կորցնում են բավականաչափ շատ ինֆորմացիա տվյալների մասին: Նկատենք, որ առաջին բաղադրիչն ունի բարձր դրական կորելացիա ձայնի բարձրության (0.875) և էներգետիկության (0.855) հետ, իսկ ակուստիկության կորելացիան բացասական է (-0.734): Արդյունքում ՀԲ1-ը կարելի է մեկնաբանել որպես **էներգետիկ և բարձր հնչողությունից դեպի ակուստիկ և հանգիստ հնչողության** առանցք:

ՀԲ2-ն ունի դրական կորելացիա դրական լիցքերի (0.593), պարայինության (0.580) և ակուստիկության (0.409) հետ, իսկ տևողության (-0.488), ինստրումենտալության (-0.415) և էներգետիկության (-0.342) կորելացիաները բացասական են: Մատչելի լեզվով

ասված՝ այստեղ հիմնականում առանձնացվում են ավելի կարճ, պարային և դրական երգերը ավելի երկար, գործիքային և ինտենսիվ երգերից: Աղյուսակից նաև երևում է, որ բացատրված վարիացիաները սահուն կերպով նվազում են, ինչը նշանակում է որ տվյալները խիստ կախվածություն չունեն որևէ կոմպոնենտից:

8 Կորելացիոն Շրջան

Նկար 4-ի կորելացիոն շրջանում նույն ուղղությամբ սլաքները ցույց են տալիս դրական կապ, իսկ հակառակ ուղղությամբ սլաքները՝ բացասական կապ: Էներգետիկության և բարձրության սլաքները մոտ են և երկար, ինչը հաստատում է նրանց ուժեղ կապը և կարևորությունը ՀԲ1-ի համար: Ակուստիկությունը ուղղված է հակառակ կողմ, իսկ պարայինությունն ու դրական հնչողությունը՝ նույն կողմ: Հանրաճանաչության սլաքը կարճ է և մոտ է կենտրոնին, այսինքն՝ առաջին երկու բաղադրիչները այն լավ չեն ներկայացնում: Սա համընկնում է թույլ կորելացիաների և ռեգրեսիայի ցածր R^2 -ի հետ:

9 Պրոյեկցիա Առաջին Երկու Հիմնական Բաղադրիչների Վրա

Նկար 5-ում ներկայացված է 10,000 երգից պատահական նմուշ, որի գույնը ցույց է տալիս ակուստիկությունը: Բարձր ակուստիկությամբ երգերը կենտրոնացած են ՀԲ1-ի ձախ կողմում, իսկ ցածր ակուստիկությամբ երգերը՝ հիմնականում աջ կողմում: Սա հաստատում է ՀԲ1-ի մեկնաբանությունը որպես ակուստիկից դեպի Էներգետիկ հնչողության մետրիկա: Պրոյեկցիոն ամպը լայն է՝ ոչ հստակ առանձնացված խմբերով: Հավանաբար այն ցույց է տալիս երաժշտական ոճերի տարբերությունները, սակայն միայն առաջին երկու բաղադրիչներով ժանրերի խիստ բաժանում անել չի կարելի, քանի որ դրանք բացատրում են տատանման միայն 40.21%-ը:

10 Եզրակացություն

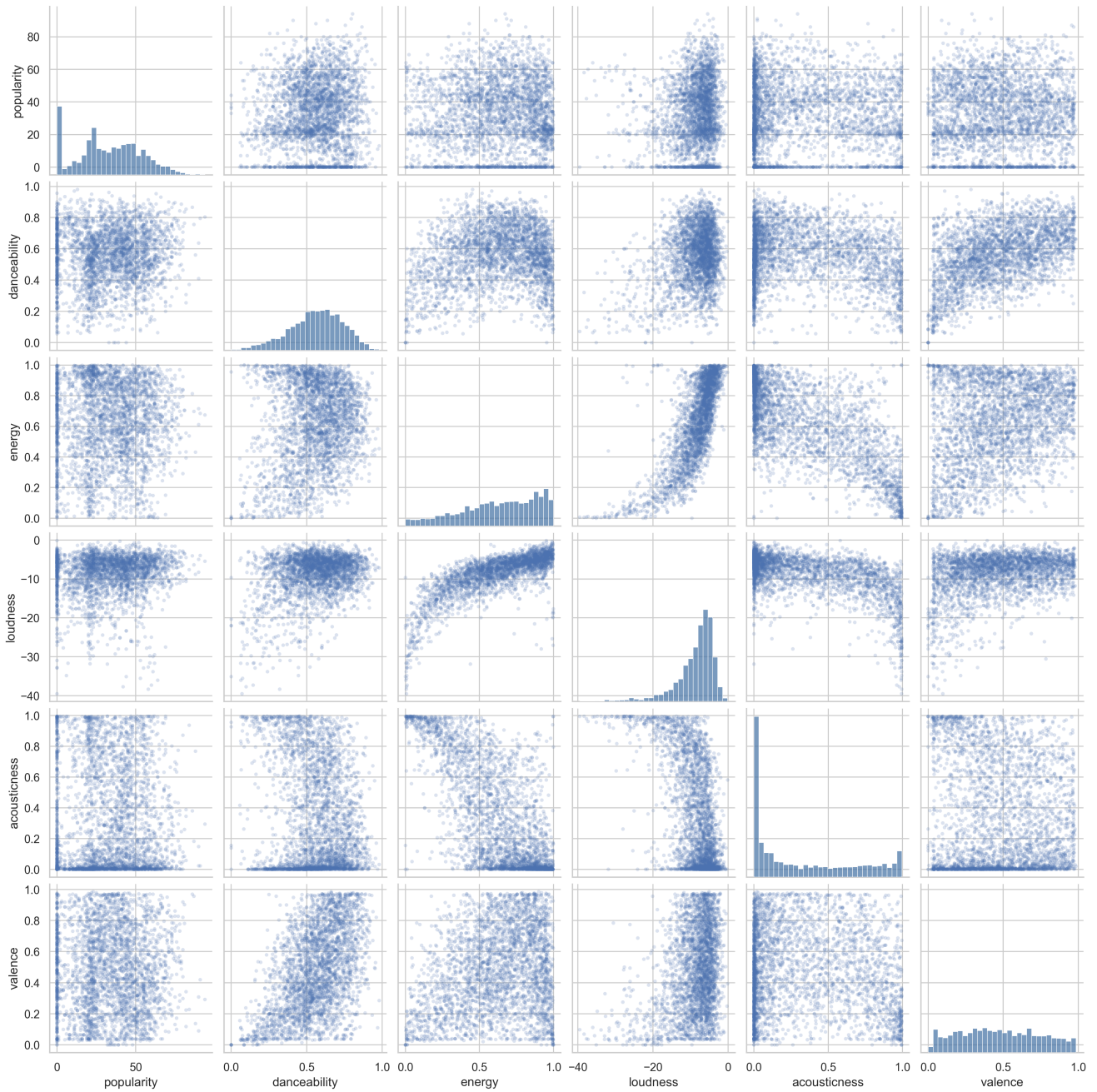
Վերլուծության արդյունքում բացահայտվեցին Spotify տվյալների երաժշտական հատկանիշների միջև հստակ կապեր: Էներգետիկությունն ու բարձրությունը ուժեղ դրական կորելացված են, իսկ ակուստիկությունը երկուսի հետ էլ ունի ուժեղ բացասական կապ: Պարայինությունն ու դրական լիցքերը նույնպես հակված են միասին աճելու: Հանրաճանաչությունը, սակայն, ձայնային հատկանիշների հետ թույլ կապեր ունի:

Ռեգրեսիոն մոդելը բացատրում է հանրաճանաչության տատանման միայն մոտ 3.1%-ը, ուստի միայն ձայնային հատկանիշները բավարար չեն ունկնդիրների նախընտրությունը կանխատեսելու համար:

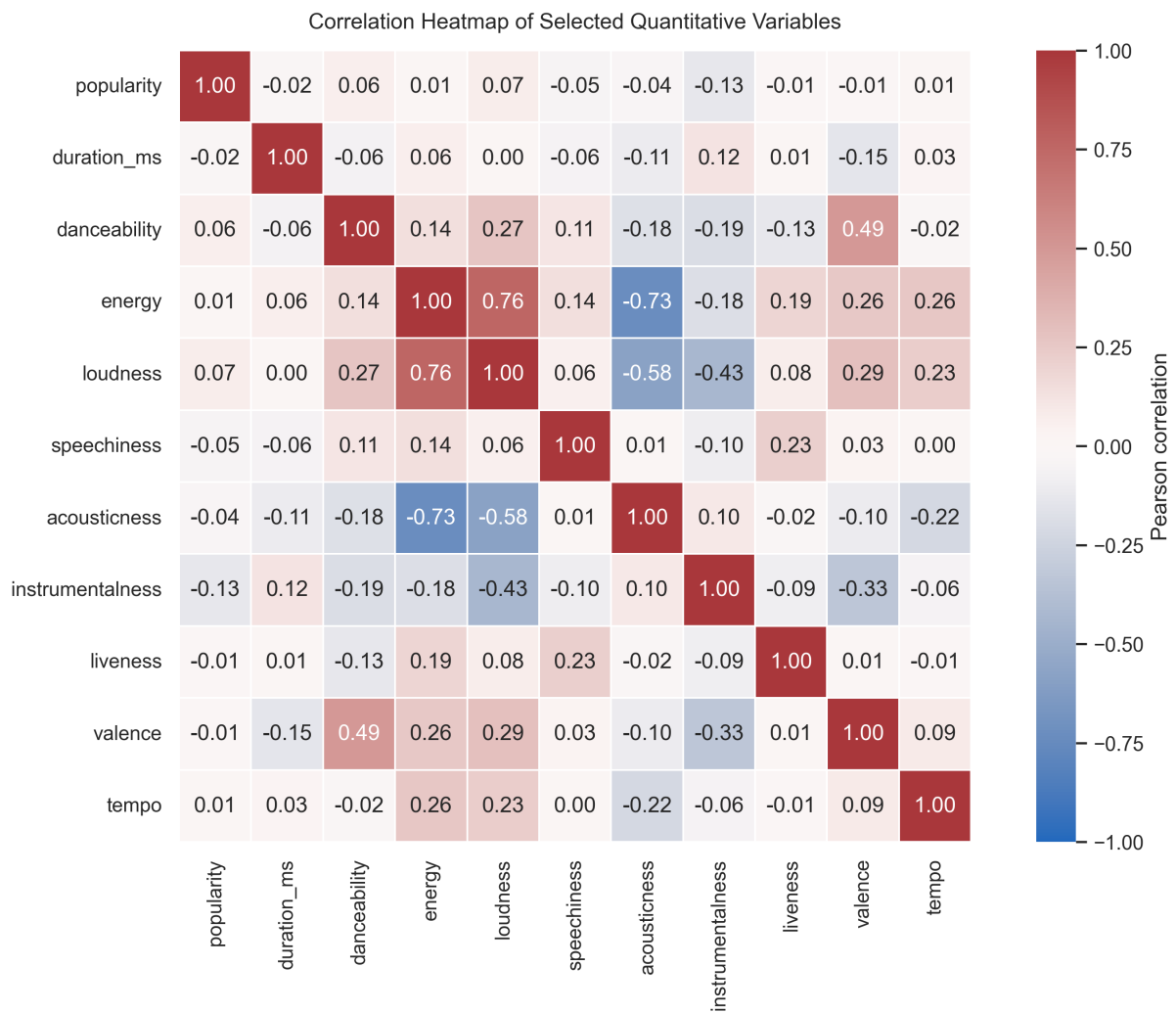
ՀԲՎ հիմնական կառուցվածքը ներկայացրեց որպես Էներգետիկ/բարձր և ակուստիկ/հանգիստ հնչողությունների հակադրություն՝ ցույց տալով բազմաչափ մոտեցման օգտակարությունը և միայն ձայնային չափումներով հանրաճանաչությունը բացատրելու սահմանափակումները:

A Նկարներ

Pairwise Scatter Plots of Selected Spotify Variables

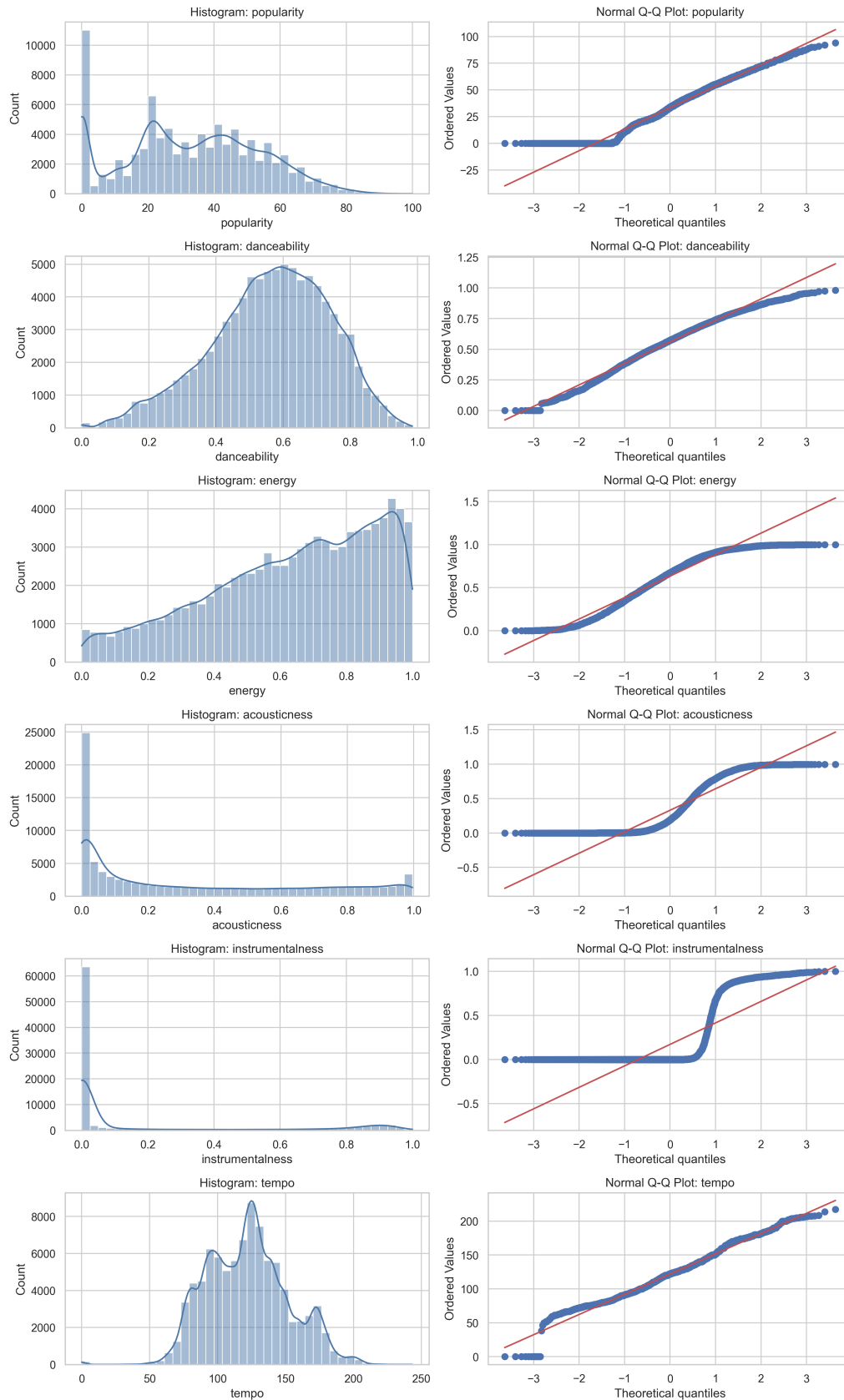


Նկար 1: Spotify-ի ներկայացուցական փոփոխականների զույգային ցրման գրաֆիկները:

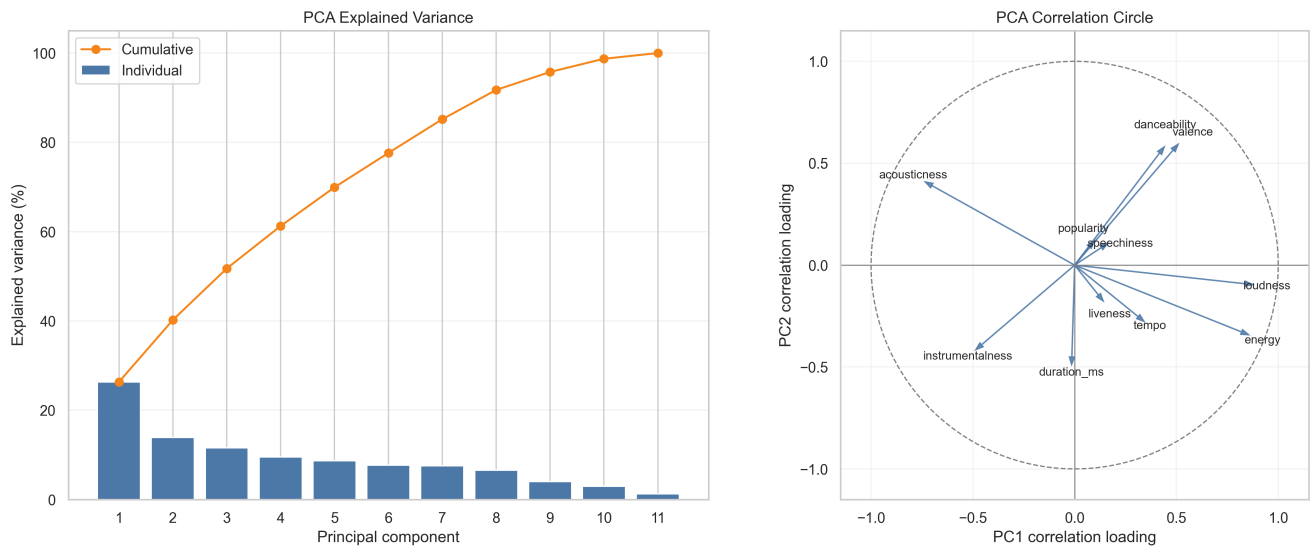


Նկար 2: Ընտրված բոլոր քանակական փոփոխականների կորելացիոն ջերմային քարտեզը:

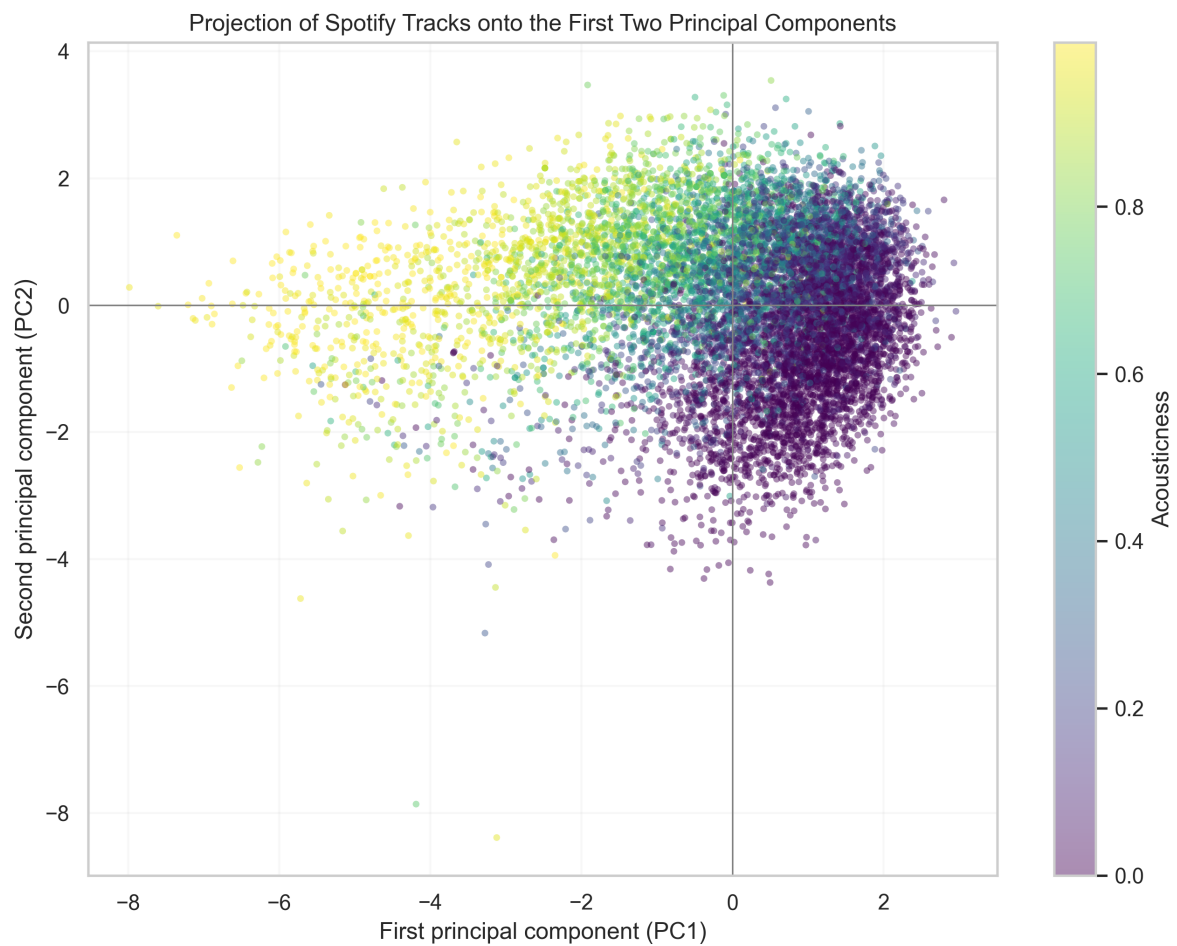
Gaussian Distribution Diagnostics



Նկար 3: Ներկայացուցչական փոփոխականների հիստոգրամները և նորմալ Q-Q գրաֆիկները:



Նկար 4: ՀԲՎ-ի բացատրած վարիացիան և կորելացիոն շրջանը:



Նկար 5: Երգերի պատահական նմուշի պրոյեկցիան ՀԲ1-ի և ՀԲ2-ի վրա:

B Python Իրականացում

Ծրագրի ամբողջական կոդը հասանելի է GitHub-ում.

<https://github.com/albpiliposyan/spotify-multivariate-analysis>

```
1 from pathlib import Path
2
3 import matplotlib
4
5 matplotlib.use("Agg")
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import pandas as pd
10 import seaborn as sns
11 import statsmodels.api as sm
12 from scipy import stats
13 from sklearn.decomposition import PCA
14 from sklearn.preprocessing import StandardScaler
15
16
17 BASE_DIR = Path(__file__).resolve().parent
18 DATA_PATH = BASE_DIR / "dataset" / "spotify_dataset.csv"
19 OUTPUT_DIR = BASE_DIR / "output"
20 FIGURE_DIR = OUTPUT_DIR / "figures"
21 TABLE_DIR = OUTPUT_DIR / "tables"
22
23 RANDOM_STATE = 42
24 PAIRPLOT_SAMPLE_SIZE = 3_000
25 QQ_SAMPLE_SIZE = 5_000
26 PROJECTION_SAMPLE_SIZE = 10_000
27
28 SELECTED_VARIABLES = [
29     "popularity",
30     "duration_ms",
31     "danceability",
32     "energy",
33     "loudness",
34     "speechiness",
35     "acousticness",
36     "instrumentalness",
37     "liveness",
38     "valence",
39     "tempo",
40 ]
41
42 PREDICTORS = [
43     "duration_ms",
44     "danceability",
45     "energy",
46     "loudness",
47     "speechiness",
48     "acousticness",
49     "instrumentalness",
50     "liveness",
51     "valence",
52     "tempo",
53 ]
54
55 PAIRPLOT_VARIABLES = [
56     "popularity",
57     "danceability",
```

```

58     "energy",
59     "loudness",
60     "acousticness",
61     "valence",
62 ]
63
64 GAUSSIAN_DIAGNOSTIC_VARIABLES = [
65     "popularity",
66     "danceability",
67     "energy",
68     "acousticness",
69     "instrumentalness",
70     "tempo",
71 ]
72
73
74 def prepare_output_directories() -> None:
75     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
76     TABLE_DIR.mkdir(parents=True, exist_ok=True)
77
78
79 def load_and_clean_data() -> tuple[pd.DataFrame, dict[str, int]]:
80     raw = pd.read_csv(DATA_PATH)
81     raw = raw.drop(columns=["Unnamed: 0"], errors="ignore")
82
83     cleaning_summary = {
84         "original_rows": len(raw),
85         "original_columns": raw.shape[1],
86         "duplicate_track_ids": int(raw.duplicated(subset="track_id").sum()),
87         "missing_selected_values": int(raw[SELECTED_VARIABLES].isna().sum().sum()),
88         "genre_count": int(raw["track_genre"].nunique()),
89     }
90
91     data = (
92         raw.drop_duplicates(subset="track_id")
93         .dropna(subset=SELECTED_VARIABLES)
94         .copy()
95     )
96     cleaning_summary["nonpositive_duration_rows"] = int(
97         (data["duration_ms"] <= 0).sum()
98     )
99     data = data.loc[data["duration_ms"] > 0].copy()
100     cleaning_summary["analysis_rows"] = len(data)
101
102     return data, cleaning_summary
103
104
105 def save_descriptive_tables(data: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
106     descriptive = (
107         data[SELECTED_VARIABLES]
108         .describe()
109         .T[["mean", "std", "min", "25%", "50%", "75%", "max"]]
110     )
111     descriptive.index.name = "variable"
112     descriptive.to_csv(TABLE_DIR / "descriptive_statistics.csv")
113
114     distribution_summary = pd.DataFrame(
115         {
116             "skewness": data[SELECTED_VARIABLES].skew(),
117             "excess_kurtosis": data[SELECTED_VARIABLES].kurtosis(),
118         }
119     )
120     distribution_summary.index.name = "variable"
121     distribution_summary.to_csv(TABLE_DIR / "distribution_shape.csv")
122

```



```

123 correlation = data[SELECTED_VARIABLES].corr()
124 correlation.to_csv(TABLE_DIR / "correlation_matrix.csv")
125
126 return descriptive, correlation
127
128
129 def create_pairplot(data: pd.DataFrame) -> None:
130     sample = data[PAIRPLOT_VARIABLES].sample(
131         n=min(PAIRPLOT_SAMPLE_SIZE, len(data)),
132         random_state=RANDOM_STATE,
133     )
134
135     grid = sns.pairplot(
136         sample,
137         diag_kind="hist",
138         plot_kws={"alpha": 0.20, "s": 12, "edgecolor": "none"},
139         diag_kws={"bins": 30, "color": "#4C78A8"},
140     )
141     grid.figure.suptitle(
142         "Pairwise Scatter Plots of Selected Spotify Variables",
143         y=1.02,
144         fontsize=15,
145     )
146     grid.figure.savefig(
147         FIGURE_DIR / "01_pairwise_scatterplots.png",
148         dpi=300,
149         bbox_inches="tight",
150     )
151     plt.close(grid.figure)
152
153
154 def create_correlation_heatmap(correlation: pd.DataFrame) -> None:
155     plt.figure(figsize=(11, 8))
156     sns.heatmap(
157         correlation,
158         cmap="vlag",
159         center=0,
160         vmin=-1,
161         vmax=1,
162         annot=True,
163         fmt=".2f",
164         square=True,
165         linewidths=0.5,
166         cbar_kws={"label": "Pearson correlation"},
167     )
168     plt.title("Correlation Heatmap of Selected Quantitative Variables", pad=12)
169     plt.tight_layout()
170     plt.savefig(
171         FIGURE_DIR / "02_correlation_heatmap.png",
172         dpi=300,
173         bbox_inches="tight",
174     )
175     plt.close()
176
177
178 def create_gaussian_diagnostics(data: pd.DataFrame) -> None:
179     qq_sample = data[GAUSSIAN_DIAGNOSTIC_VARIABLES].sample(
180         n=min(QQ_SAMPLE_SIZE, len(data)),
181         random_state=RANDOM_STATE,
182     )
183
184     fig, axes = plt.subplots(
185         nrows=len(GAUSSIAN_DIAGNOSTIC_VARIABLES),
186         ncols=2,
187         figsize=(12, 20),

```

```

188 )
189
190 for row, variable in enumerate(GAUSSIAN_DIAGNOSTIC_VARIABLES):
191     sns.histplot(
192         data=data,
193         x=variable,
194         bins=40,
195         kde=True,
196         color="#4C78A8",
197         ax=axes[row, 0],
198     )
199     axes[row, 0].set_title(f"Histogram: {variable}")
200
201     stats.probplot(
202         qq_sample[variable],
203         dist="norm",
204         plot=axes[row, 1],
205     )
206     axes[row, 1].set_title(f"Normal Q-Q Plot: {variable}")
207
208     fig.suptitle(
209         "Gaussian Distribution Diagnostics",
210         fontsize=16,
211         y=1.002,
212     )
213     fig.tight_layout()
214     fig.savefig(
215         FIGURE_DIR / "03_gaussian_diagnostics.png",
216         dpi=300,
217         bbox_inches="tight",
218     )
219     plt.close(fig)
220
221
222 def fit_standardized_regression(
223     data: pd.DataFrame,
224 ) -> tuple[pd.DataFrame, pd.Series, StandardScaler, sm.regression.linear_model.RegressionResultsWrapper
225     ]:
226     scaler = StandardScaler()
227     standardized_predictors = pd.DataFrame(
228         scaler.fit_transform(data[PREDICTORS]),
229         columns=PREDICTORS,
230         index=data.index,
231     )
232     response = data["popularity"]
233
234     model = sm.OLS(
235         response,
236         sm.add_constant(standardized_predictors, has_constant="add"),
237     ).fit()
238
239     return standardized_predictors, response, scaler, model
240
241
242 def model_table(
243     model: sm.regression.linear_model.RegressionResultsWrapper,
244 ) -> pd.DataFrame:
245     confidence_intervals = model.conf_int()
246     table = pd.DataFrame(
247         {
248             "coefficient": model.params,
249             "standard_error": model.bse,
250             "t_statistic": model.tvalues,
251             "p_value": model.pvalues,
252             "ci_2.5%": confidence_intervals[0],

```

```

252         "ci_97.5%": confidence_intervals[1],
253     }
254 )
255 table.index.name = "variable"
256 return table
257
258
259 def backward_elimination(
260     predictors: pd.DataFrame,
261     response: pd.Series,
262     alpha: float = 0.05,
263 ) -> tuple[
264     list[str],
265     sm.regression.linear_model.RegressionResultsWrapper,
266     pd.DataFrame,
267 ]:
268     selected = list(predictors.columns)
269     history: list[dict[str, float | int | str]] = []
270     step = 1
271
272     while True:
273         model = sm.OLS(
274             response,
275             sm.add_constant(predictors[selected], has_constant="add"),
276         ).fit()
277         p_values = model.pvalues.drop("const")
278         largest_p_value = float(p_values.max())
279
280         if largest_p_value <= alpha:
281             break
282
283         removed_variable = str(p_values.idxmax())
284         history.append(
285             {
286                 "step": step,
287                 "removed_variable": removed_variable,
288                 "p_value_at_removal": largest_p_value,
289             }
290         )
291         selected.remove(removed_variable)
292         step += 1
293
294     history_table = pd.DataFrame(history)
295     return selected, model, history_table
296
297
298 def save_regression_results(
299     predictors: pd.DataFrame,
300     response: pd.Series,
301     full_model: sm.regression.linear_model.RegressionResultsWrapper,
302 ) -> tuple[
303     list[str],
304     sm.regression.linear_model.RegressionResultsWrapper,
305     pd.DataFrame,
306 ]:
307     full_table = model_table(full_model)
308     full_table.to_csv(TABLE_DIR / "regression_full_model.csv")
309     (TABLE_DIR / "regression_full_model_summary.txt").write_text(
310         full_model.summary().as_text(),
311         encoding="utf-8",
312     )
313
314     selected_variables, selected_model, history = backward_elimination(
315         predictors,
316         response,

```

```

317     alpha=0.05,
318 )
319 model_table(selected_model).to_csv(TABLE_DIR / "regression_selected_model.csv")
320 history.to_csv(TABLE_DIR / "backward_elimination_history.csv", index=False)
321 (TABLE_DIR / "regression_selected_model_summary.txt").write_text(
322     selected_model.summary().as_text(),
323     encoding="utf-8",
324 )
325
326 return selected_variables, selected_model, history
327
328
329 def orient_pca_axes(
330     components: np.ndarray,
331     scores: np.ndarray,
332     variable_names: list[str],
333 ) -> tuple[np.ndarray, np.ndarray]:
334     oriented_components = components.copy()
335     oriented_scores = scores.copy()
336
337     energy_index = variable_names.index("energy")
338     valence_index = variable_names.index("valence")
339
340     if oriented_components[0, energy_index] < 0:
341         oriented_components[0, :] *= -1
342         oriented_scores[:, 0] *= -1
343
344     if oriented_components[1, valence_index] < 0:
345         oriented_components[1, :] *= -1
346         oriented_scores[:, 1] *= -1
347
348     return oriented_components, oriented_scores
349
350
351 def run_pca(
352     data: pd.DataFrame,
353 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
354     scaler = StandardScaler()
355     standardized_data = scaler.fit_transform(data[SELECTED_VARIABLES])
356
357     pca = PCA()
358     raw_scores = pca.fit_transform(standardized_data)
359     components, scores = orient_pca_axes(
360         pca.components_,
361         raw_scores,
362         SELECTED_VARIABLES,
363     )
364
365     explained_variance = pd.DataFrame(
366         {
367             "principal_component": [
368                 f"PC{i}" for i in range(1, len(SELECTED_VARIABLES) + 1)
369             ],
370             "explained_variance_ratio": pca.explained_variance_ratio_,
371             "cumulative_explained_variance": np.cumsum(
372                 pca.explained_variance_ratio_
373             ),
374         }
375     )
376     explained_variance.to_csv(
377         TABLE_DIR / "pca_explained_variance.csv",
378         index=False,
379     )
380
381     correlation_loadings = components.T * np.sqrt(pca.explained_variance_)

```

```

382     loadings = pd.DataFrame(
383         correlation_loadings,
384         index=SELECTED_VARIABLES,
385         columns=[f"PC{i}" for i in range(1, len(SELECTED_VARIABLES) + 1)],
386     )
387     loadings.index.name = "variable"
388     loadings.to_csv(TABLE_DIR / "pca_correlation_loadings.csv")
389
390     score_table = pd.DataFrame(
391         scores[:, :2],
392         columns=["PC1", "PC2"],
393         index=data.index,
394     )
395     score_table["acousticness"] = data["acousticness"]
396     score_table["track_genre"] = data["track_genre"]
397
398     create_pca_summary_figure(explained_variance, loadings)
399     create_pca_projection(score_table)
400
401     return explained_variance, loadings, score_table
402
403
404 def create_pca_summary_figure(
405     explained_variance: pd.DataFrame,
406     loadings: pd.DataFrame,
407 ) -> None:
408     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
409
410     pc_numbers = np.arange(1, len(explained_variance) + 1)
411     axes[0].bar(
412         pc_numbers,
413         explained_variance["explained_variance_ratio"] * 100,
414         color="#4C78A8",
415         label="Individual",
416     )
417     axes[0].plot(
418         pc_numbers,
419         explained_variance["cumulative_explained_variance"] * 100,
420         marker="o",
421         color="#F58518",
422         label="Cumulative",
423     )
424     axes[0].set_xticks(pc_numbers)
425     axes[0].set_xlabel("Principal component")
426     axes[0].set_ylabel("Explained variance (%)")
427     axes[0].set_title("PCA Explained Variance")
428     axes[0].legend()
429     axes[0].grid(axis="y", alpha=0.25)
430
431     circle = plt.Circle((0, 0), 1, color="gray", fill=False, linestyle="--")
432     axes[1].add_artist(circle)
433     label_offsets = {
434         "danceability": (-0.03, 0.06),
435         "valence": (0.03, 0.01),
436         "popularity": (-0.06, 0.05),
437         "speechiness": (0.05, -0.01),
438         "liveness": (0.02, -0.05),
439     }
440     for variable in loadings.index:
441         x = loadings.loc[variable, "PC1"]
442         y = loadings.loc[variable, "PC2"]
443         axes[1].arrow(
444             0,
445             0,
446             x,

```

```

447         y,
448         color="#4C78A8",
449         alpha=0.85,
450         head_width=0.025,
451         length_includes_head=True,
452     )
453     x_offset, y_offset = label_offsets.get(variable, (0.0, 0.0))
454     axes[1].text(
455         x * 1.08 + x_offset,
456         y * 1.08 + y_offset,
457         variable,
458         fontsize=9,
459         ha="center",
460         va="center",
461     )
462
463     axes[1].axhline(0, color="gray", linewidth=0.8)
464     axes[1].axvline(0, color="gray", linewidth=0.8)
465     axes[1].set_xlim(-1.15, 1.15)
466     axes[1].set_ylim(-1.15, 1.15)
467     axes[1].set_aspect("equal", adjustable="box")
468     axes[1].set_xlabel("PC1 correlation loading")
469     axes[1].set_ylabel("PC2 correlation loading")
470     axes[1].set_title("PCA Correlation Circle")
471     axes[1].grid(alpha=0.20)
472
473     fig.tight_layout()
474     fig.savefig(
475         FIGURE_DIR / "04_pca_explained_variance_and_correlation_circle.png",
476         dpi=300,
477         bbox_inches="tight",
478     )
479     plt.close(fig)
480
481
482 def create_pca_projection(score_table: pd.DataFrame) -> None:
483     projection_sample = score_table.sample(
484         n=min(PROJECTION_SAMPLE_SIZE, len(score_table)),
485         random_state=RANDOM_STATE,
486     )
487
488     plt.figure(figsize=(9, 7))
489     scatter = plt.scatter(
490         projection_sample["PC1"],
491         projection_sample["PC2"],
492         c=projection_sample["acousticness"],
493         cmap="viridis",
494         s=12,
495         alpha=0.45,
496         edgecolors="none",
497     )
498     plt.colorbar(scatter, label="Acousticness")
499     plt.axhline(0, color="gray", linewidth=0.8)
500     plt.axvline(0, color="gray", linewidth=0.8)
501     plt.xlabel("First principal component (PC1)")
502     plt.ylabel("Second principal component (PC2)")
503     plt.title("Projection of Spotify Tracks onto the First Two Principal Components")
504     plt.grid(alpha=0.15)
505     plt.tight_layout()
506     plt.savefig(
507         FIGURE_DIR / "05_pca_projection.png",
508         dpi=300,
509         bbox_inches="tight",
510     )
511     plt.close()

```

```

512
513
514 def strongest_correlations(correlation: pd.DataFrame, n: int = 10) -> pd.DataFrame:
515     mask = np.triu(np.ones_like(correlation, dtype=bool), k=1)
516     pairs = correlation.where(mask).stack().reset_index()
517     pairs.columns = ["variable_1", "variable_2", "correlation"]
518     pairs["absolute_correlation"] = pairs["correlation"].abs()
519     return pairs.sort_values("absolute_correlation", ascending=False).head(n)
520
521
522 def write_analysis_summary(
523     cleaning_summary: dict[str, int],
524     descriptive: pd.DataFrame,
525     correlation: pd.DataFrame,
526     full_model: sm.regression.linear_model.RegressionResultsWrapper,
527     selected_variables: list[str],
528     selected_model: sm.regression.linear_model.RegressionResultsWrapper,
529     elimination_history: pd.DataFrame,
530     explained_variance: pd.DataFrame,
531     loadings: pd.DataFrame,
532 ) -> None:
533     top_correlations = strongest_correlations(correlation, n=10)
534     top_correlations.to_csv(TABLE_DIR / "strongest_correlations.csv", index=False)
535
536     lines = [
537         "SPOTIFY MULTIVARIATE ANALYSIS SUMMARY",
538         "",
539         "DATA CLEANING",
540         f"Original rows: {cleaning_summary['original_rows']}",
541         f"Duplicate track IDs removed: {cleaning_summary['duplicate_track_ids']}",
542         f"Missing selected values: {cleaning_summary['missing_selected_values']}",
543         (
544             "Nonpositive-duration rows removed: "
545             f"{cleaning_summary['nonpositive_duration_rows']}"
546         ),
547         f"Rows used in analysis: {cleaning_summary['analysis_rows']}",
548         f"Genre labels in original data: {cleaning_summary['genre_count']}",
549         "",
550         "SELECTED VARIABLE MEANS",
551         descriptive["mean"].round(4).to_string(),
552         "",
553         "STRONGEST CORRELATIONS",
554         top_correlations.round(4).to_string(index=False),
555         "",
556         "FULL STANDARDIZED REGRESSION",
557         f"R-squared: {full_model.rsquared:.6f}",
558         f"Adjusted R-squared: {full_model.rsquared_adj:.6f}",
559         model_table(full_model)[["coefficient", "p_value"].round(6).to_string(),
560         "",
561         "BACKWARD ELIMINATION",
562         (
563             elimination_history.to_string(index=False)
564             if not elimination_history.empty
565             else "No variables were removed at alpha = 0.05."
566         ),
567         f"Selected variables: {' '.join(selected_variables)}",
568         f"Selected model R-squared: {selected_model.rsquared:.6f}",
569         f"Selected model adjusted R-squared: {selected_model.rsquared_adj:.6f}",
570         "",
571         "PCA EXPLAINED VARIANCE",
572         explained_variance.head(6).round(6).to_string(index=False),
573         "",
574         "PCA CORRELATION LOADINGS FOR PC1 AND PC2",
575         loadings[["PC1", "PC2"].round(4).to_string(),
576         ""

```

```

577 ]
578 (OUTPUT_DIR / "analysis_summary.txt").write_text(
579     "\n".join(lines),
580     encoding="utf-8",
581 )
582
583
584 def main() -> None:
585     prepare_output_directories()
586     sns.set_theme(style="whitegrid", context="notebook")
587
588     data, cleaning_summary = load_and_clean_data()
589     descriptive, correlation = save_descriptive_tables(data)
590
591     create_pairplot(data)
592     create_correlation_heatmap(correlation)
593     create_gaussian_diagnostics(data)
594
595     predictors, response, _, full_model = fit_standardized_regression(data)
596     selected_variables, selected_model, elimination_history = save_regression_results(
597         predictors,
598         response,
599         full_model,
600     )
601
602     explained_variance, loadings, _ = run_pca(data)
603
604     write_analysis_summary(
605         cleaning_summary,
606         descriptive,
607         correlation,
608         full_model,
609         selected_variables,
610         selected_model,
611         elimination_history,
612         explained_variance,
613         loadings,
614     )
615
616     print(f"Analysis complete. Results saved to: {OUTPUT_DIR}")
617     print(f"Rows used in analysis: {cleaning_summary['analysis_rows']}")
618     print(f"Full regression R-squared: {full_model.rsquared:.4f}")
619     print(f"Selected variables: {' ', ' '.join(selected_variables)}")
620     print(
621         "PC1 and PC2 cumulative explained variance: "
622         f"{explained_variance.loc[1, 'cumulative_explained_variance']:.4f}"
623     )
624
625
626 if __name__ == "__main__":
627     main()

```